

PADRONIZAÇÃO DE DEVOPS: DEPLOY AUTOMÁTICO (CONTINUOUS DEPLOYMENT)

Projeto: Sistema de Estações Meteorológicas IoT — Cenário 01 (Agritech)
Instituição / Parceiro: Fatec São José dos Campos / Tecsus — Tecnologias para a Sustentabilidade
Curso / Semestre: 4º DSM — Semestre 2026-2
Frente: Engenharia de DevOps e Deploy

1. OBJETIVO E ESCOPO

Este documento estabelece as diretrizes técnicas e operacionais para a automação do processo de Deploy Contínuo (CD - Continuous Deployment) do projeto. A padronização tem como finalidade assegurar que alterações integradas e aprovadas no repositório sejam publicadas no ambiente de produção de forma consistente, previsível e sem intervenção manual.

1.1. Requisitos Não Funcionais Atendidos

- RNF02 (Disponibilidade mínima de 99%): Garantida através de serviços gerenciados com reinício automático e verificação de integridade operacional.
- RNF09 (Padrões de Desenvolvimento e Manutenibilidade): Pipeline uniforme de automação documentado e versionado no repositório.
- RNF12 (API Disponível 24x7): Deploy com downtime reduzido e monitoramento de disponibilidade pós-implantação.

2. MATRIZ DE RESPONSABILIDADES DA EQUIPE

O fluxo de entrega contínua depende da estrita observância da divisão de responsabilidades entre as três frentes:

Frente	Responsável	Entregáveis Obrigatórios	Condição de Bloqueio
Testes	Colega de Testes	Suíte de testes unitários e de integração (pytest para Backend e vitest para Frontend). Endpoint /health validado.	Falha de execução nos testes locais ou ausência de cobertura mínima acordada.
Integração Contínua (CI)	Colega de CI	Workflow .github/workflows/ci.yml configurado para executar checagem de lint (ruff), checagem estática de tipos (tsc), testes automatizados e compilação de build a cada Pull Request.	PR reprovado automaticamente se houver qualquer erro de lint, tipagem ou teste quebrado.
Deploy Contínuo (CD)	Responsável por Deploy	Workflow .github/workflows/deploy.yml, configuração de provedores em nuvem, gestão de segredos, smoke test pós-deploy e rotinas de contingência/rollback.	O deploy é impedido de executar se os jobs de CI não obtiverem status de aprovação completa na branch main.

2.1. Contrato Técnico de Integração

- Imutabilidade da branch principal: A branch main reflete exclusivamente o estado operacional de produção. Commits diretos são bloqueados; todas as alterações chegam via Pull Request aprovado.
- Health Check Obrigatório: O serviço de backend deve obrigatoriamente manter a rota GET /health ativa, retornando código HTTP 200 e confirmação de conectividade com as camadas de persistência.

- Configuração Externa por Ambiente: Nenhuma credencial, URL ou porta de conexão deve estar codificada no código-fonte. Todo parâmetro sensível ou dependente de ambiente deve ser carregado via variáveis de ambiente.

3. ARQUITETURA DE AMBIENTES E DESTINOS DE DEPLOY

A infraestrutura é distribuída entre serviços gerenciados e provedores de nuvem especializados para garantir o isolamento das camadas do sistema:

Componente	Diretório no Monorepo	Provedor de Produção	Estratégia de Deploy
Frontend (Portal Web)	/frontend	Vercel	Build estático otimizado (React 18 + Vite + Tailwind CSS) distribuído via CDN global. Preview automático em PRs e produção na main.
Backend (API / Ingestão)	/backend	Render / Railway	Contêiner Docker executando FastAPI com servidor ASGI Uvicorn. Acionamento via Deploy Hook autenticado.
Banco de Dados de Telemetria	—	MongoDB Atlas	Cluster gerenciado com suporte nativo a Time Series Collections e particionamento temporal.
Mensageria e Cache	—	Redis Managed (Render / Upstash)	Armazenamento em memória para dados recentes de telemetria, controle de rate limit e mecanismo Pub/Sub.
Broker IoT (MQTT)	/mosquitto (apenas dev)	HiveMQ Cloud	Broker MQTT gerenciado para produção com suporte a TLS e autenticação por credenciais com QoS 1.

4. GERENCIAMENTO DE SEGREDOS E VARIÁVEIS DE AMBIENTE

As credenciais necessárias para operação dos pipelines e comunicação entre os serviços devem ser cadastradas exclusivamente nas plataformas de gestão de segredos adequadas.

4.1. Segredos do Repositório (GitHub Actions Secrets)

Nome da Variável	Finalidade	Escopo de Acesso
RENDER_BACKEND_DEPLOY_HOOKURL	URL de acionamento do Deploy Hook do serviço de backend	Pipeline de Deploy (deploy.yml)
VERCEL_TOKEN	Token de autorização para comandos via Vercel CLI	Pipeline de Deploy (deploy.yml)
VERCEL_ORG_ID	Identificador da organização no Vercel	Pipeline de Deploy (deploy.yml)
VERCEL_PROJECT_ID	Identificador do projeto do frontend no Vercel	Pipeline de Deploy (deploy.yml)
PROD_API_URL	URL base do backend em produção para execução do smoke test	Pipeline de Deploy (deploy.yml)
PROD_FRONT_URL	URL base do frontend em produção para verificação de resposta HTTP	Pipeline de Deploy (deploy.yml)

4.2. Variáveis Injetadas no Ambiente de Execução do Backend

- DATABASE_URL: URI de conexão autenticada com o cluster MongoDB Atlas.

- REDIS_URL: URI de conexão ao serviço Redis gerenciado.
- MQTT_BROKER_HOST, MQTT_BROKER_PORT, MQTT_USERNAME, MQTT_PASSWORD: Parâmetros de autenticação no HiveMQ Cloud.
- JWT_SECRET_KEY, JWT_ALGORITHM: Parâmetros criptográficos para emissão e validação de tokens JWT.
- CORS_ORIGINS: Lista restrita de domínios autorizados do frontend em produção.

5. PIPELINE DE DEPLOY AUTOMÁTICO

O arquivo de configuração do workflow de publicação em produção deve residir em `.github/workflows/deploy.yml`.

```
name: Continuous Deployment (Producao)

on:
  push:
    branches:
      - main

concurrency:
  group: production-deployment
  cancel-in-progress: false

jobs:
  verify-prerequisites:
    name: Validacao de Pre-requisitos
    runs-on: ubuntu-latest
    steps:
      - name: Confirmar branch main
        run: echo "Iniciando pipeline de entrega continua para a branch main."

  deploy-backend:
    name: Deploy Backend (FastAPI / Render)
    needs: verify-prerequisites
    runs-on: ubuntu-latest
    steps:
      - name: Acionar Deploy Hook
        run: |
          echo "Disparando requisicao de deploy no Render..."
          HTTP_STATUS=$(curl -s -o /dev/null -w "%{http_code}" -X POST "${{ secrets.RENDER_BACKEND_DEPLOY_HOOK }}")
          if [ "$HTTP_STATUS" -ne 200 ] && [ "$HTTP_STATUS" -ne 201 ]; then
            echo "Falha ao acionar webhook de deploy.Codigo HTTP retornado: $HTTP_STATUS"
            exit 1
          fi
          echo "Ordem de deploy aceita com sucesso pelo provedor."

      - name: Smoke Test do Backend (Healthcheck Polling)
        run: |
          echo "Aguardando inicializacao do servico para verificacao de integridade..."
          sleep 30
          SUCCESS=0
          for i in {1..8}; do
            STATUS=$(curl -s -o /dev/null -w "%{http_code}" "${{ secrets.PROD_API_URL }}/health" || true)
            if [ "$STATUS" -eq 200 ]; then
              echo "Smoke Test concluido com sucesso. API operacional (HTTP 200)."
```

```
fi

deploy-frontend:
  name: Deploy Frontend (React / Vercel)
  needs: verify-prerequisites
  runs-on: ubuntu-latest
  steps:
    - name: Checkout do repositório
      uses: actions/checkout@v4

    - name: Instalação da Vercel CLI
      run: npm install --global vercel@latest

    - name: Obter configurações de produção da Vercel
      run: vercel pull --yes --environment=production --token=${{ secrets.VERCEL_TOKEN }}

    - name: Build do Frontend para produção
      run: vercel build --prod --token=${{ secrets.VERCEL_TOKEN }}

    - name: Publicação do build em produção
      run: vercel deploy --prebuilt --prod --token=${{ secrets.VERCEL_TOKEN }}

    - name: Smoke Test do Frontend
      run: |
        echo "Validando disponibilidade do portal web..."
        HTTP_STATUS=$(curl -s -o /dev/null -w "%{http_code}" "${{ secrets.PROD_FRONT_URL }}" || true)
        if [ "$HTTP_STATUS" -ne 200 ]; then
          echo "Falha no Smoke Test do Frontend. Código HTTP retornado: $HTTP_STATUS"
          exit 1
        fi
        echo "Frontend operacional e acessível publicamente."
```

6. PROTOCOLO DE SMOKE TEST E CRITÉRIOS DE ACEITAÇÃO

O Smoke Test é a validação pós-deploy obrigatória executada diretamente pelo pipeline de CD. Se o teste não obtiver sucesso, o pipeline reporta estado de erro e notifica a equipe.

6.1. Critérios de Aprovação

1. Backend (GET /health):
 - Resposta HTTP com código 200 em tempo inferior a 2000 milissegundos.
 - Corpo da resposta contendo status indicando serviço ativo e conexão com banco de dados ativa.
2. Frontend (GET /):
 - Resposta HTTP com código 200 e entrega do cabeçalho HTML inicial sem erros de roteamento ou carregamento de scripts estáticos.

7. PROCEDIMENTO OPERACIONAL PADRÃO (POP) DE ROLLBACK

Caso seja identificada anomalia grave em ambiente produtivo pós-implantação, a equipe deve seguir as etapas de reversão rápida.

7.1. Reversão Imediata via Plataforma (Tempo de Resolução: < 1 minuto)

- Frontend (Vercel):
 1. Acessar o dashboard do projeto no Vercel.
 2. Navegar até a aba Deployments.

3. Identificar o último deploy estável anterior.
4. Acessar o menu de contexto (...) e acionar a opção Instant Rollback. O tráfego da CDN é redirecionado imediatamente para a versão prévia estável sem necessidade de recompilação.
 - Backend (Render):
1. Acessar o painel do serviço de backend no Render.
2. Navegar até a aba Events / Deploys.
3. Selecionar o registro do deploy estável anterior.
4. Clicar em Rollback to this deploy. O contêiner anterior é restaurado e reativado.

7.2. Reversão Definitiva no Controle de Versão (Git)

Após a estabilização emergencial via painel da plataforma, deve-se alinhar o histórico da branch main:

```
# Atualizar a branch local de trabalho
git checkout main
git pull origin main

# Reverter o commit causador da inconsistência
git revert HEAD --no-edit

# Enviar a reversão para a branch principal
git push origin main
```

A emissão do push dispara automaticamente um novo ciclo de CI/CD, garantindo que o repositório e o ambiente de produção permaneçam estritamente sincronizados.

8. REGRAS DE GOVERNANÇA E OPERAÇÃO

1. Bloqueio de Commits Diretos: Nenhum membro da equipe tem permissão de realizar push diretamente nas branches main e develop.
2. Critério Mínimo de Aprovação de PR: Cada Pull Request deve obter aprovação de pelo menos um revisor e aprovação de todos os testes automatizados do CI.
3. Isolamento de Credenciais: É expressamente vedado o versionamento de arquivos contendo chaves reais, como .env ou credenciais de banco. O repositório deve conter apenas arquivos .env.example com chaves sem valor preenchido.